

# Is It a Promise?

*How to tell, from the declaration, the body, or the caller — and the full list of things that return one*

October 2026

**The one question.** For every function call you read: *ticket or value*? A Promise is a ticket for a value that arrives later; `await` cashes it. Holding a ticket and treating it as the value is the single most common planted bug.

## What makes a function return a Promise (the facts)

Exactly two things. No annotation is required for the Promise to exist.

| Fact                                                 | Example                                         | Result                                                                   |
|------------------------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------|
| The <code>async</code> keyword                       | <code>async function f() { return 5 }</code>    | <code>Promise&lt;number&gt;</code> , always, even if the body is instant |
| The body returns something that is already a Promise | <code>function f() { return fetch(url) }</code> | <code>Promise&lt;Response&gt;</code> , passed through                    |

The `: Promise<T>` annotation does not make a function return a Promise; it only documents that one of the two facts holds. It is optional and frequently omitted.

## How to tell (the clues, in order of reliability)

| # | Clue                                        | Where you look                        | Example                                                                                               |
|---|---------------------------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------|
| 1 | <code>async</code> keyword                  | the declaration                       | <code>async function getUser(...), const f = async () =&gt;</code>                                    |
| 2 | <code>: Promise&lt;T&gt;</code> return type | the declaration, after <code>)</code> | <code>function getUser(id): Promise&lt;User&gt;</code>                                                |
| 3 | the body returns a Promise-maker            | the <code>return</code> statements    | <code>return fetch(...), return db.query(...)</code>                                                  |
| 4 | callers <code>await</code> it               | every call site                       | <code>const u = await getUser(id), .then(...), inside Promise.all</code>                              |
| 5 | the name and the job                        | weakest; a hint only                  | <code>fetch / load / save / query / read / write / send / connect</code> → usually <code>async</code> |

One clue is enough. Clue 1 is certain. Clue 2 is certain when present. Clue 3 needs you to chase the `return`. Clue 4 is the author telling you. Clue 5 only tells you where to look.

**Say, when you cannot tell:** “Assuming `getUser` is `async`, as the `await` on line 12 suggests; if it is synchronous, line 20’s `await` is harmless but line 31’s missing one is not.” An honest conditional beats a wrong certainty.

## Clue 3 in detail: the pass-through

```
function getUser(id: string) {  
  return fetch(`/users/${id}`);  
}
```

No `async`, no annotation, still a Promise. Say: “`getUser` returns whatever `fetch` returns.” Then: “`fetch` returns `Promise<Response>`.” So `getUser` returns `Promise<Response>`. The function is a pipe: whatever goes into the `return` comes out of the call.

**return X means the function returns the type of X.**

| Body                                                  | X is                                               | Function returns                                                   |
|-------------------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------|
| <code>return 5</code>                                 | a number                                           | <code>number</code>                                                |
| <code>return fetch(url)</code>                        | <code>Promise&lt;Response&gt;</code>               | <code>Promise&lt;Response&gt;</code>                               |
| <code>return db.query(sql)</code>                     | <code>Promise&lt;Row[]&gt;</code>                  | <code>Promise&lt;Row[]&gt;</code>                                  |
| <code>return getUser(id)</code>                       | whatever <code>getUser</code> returns              | the same; chase one more step                                      |
| <code>return fetch(url).then(r =&gt; r.json())</code> | a Promise ( <code>.then</code> always gives one)   | <code>Promise&lt;any&gt;</code>                                    |
| <code>return new Promise(...)</code>                  | a Promise                                          | a Promise                                                          |
| <code>return items.map(f)</code>                      | an array                                           | an array, not a Promise                                            |
| <code>return await fetch(url)</code>                  | the unwrapped Response (needs <code>async</code> ) | <code>Promise&lt;Response&gt;</code> because of <code>async</code> |

The chase: follow `return` until you hit an `async` keyword, a known Promise-maker from the list below, or a plain value. Stop there.

**Why authors write it:** `async function f() { return await fetch(...) }` and `function f() { return fetch(...) }` do the same job; the second is shorter. It is a legitimate style, not a finding.

**The one trap in that style:**

```
function getUser(id: string) {
  try {
    return fetch(`/users/${id}`); // the Promise leaves the try before it settles
  } catch (e) {
    log(e); // never runs for a network failure
  }
}
```

A rejection escapes the `try` because nothing waited inside it. This is the one place `async` + `return await` genuinely matters.

**The corrections.** Two, depending on whether `getUser` should handle the error itself or hand it to the caller.

*Fix A, catch it here: make the function `async` and `await` inside the `try`.*

```
here
async function getUser(id: string) {
  try {
    return await fetch(`/users/${id}`); // waits INSIDE the try; a rejection is thrown right
  } catch (e) {
    log(e); // now runs on a network failure
    throw e; // re-throw (or return a fallback); never swallow
  }
}
silently
```

Three changes: `async` on the function, `await` before `fetch`, and something after `log(e)` so the error is not eaten. `return await` is the one combination where the `await` is not redundant.

*Fix B, do not catch it here: delete the `try` and let the caller handle it.*

```
function getUser(id: string) {
  return fetch(`/users/${id}`); // honest pass-through; nothing to catch here
}

// the caller:
try {
  const res = await getUser("42"); // the await is inside THIS try, so it catches
} catch (e) {
  log(e);
}
```

*Which to recommend:* B when the function has nothing useful to do with the error (pure pass-through); A when it should log, add context, retry, or return a default.

*Say it as:* “Line 3 returns the Promise without `await`, so a rejection escapes the `try` and `log` never runs; either `async` + `return await`, or drop the `try` and let the caller handle it.”

*What neither fix covers:* `fetch` resolves on a 404 or 500, so the `catch` only sees network failures. If the requirement is “handle a missing user”, add `if (!res.ok) throw new Error(...)` after the `await`, inside the `try`:

```
async function getUser(id: string) {
  try {
    const res = await fetch(`/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`); // 404/500 become errors too
    return res;
  } catch (e) {
    log(e);
    throw e;
  }
}
```

# Everything that returns a Promise

Memorize these so the chase can stop when it reaches one.

## Language and standard library

| Call                                                       | Returns                                                | Notes                                                                              |
|------------------------------------------------------------|--------------------------------------------------------|------------------------------------------------------------------------------------|
| any function marked <code>async</code>                     | <code>Promise&lt;whatever it returns&gt;</code>        | the keyword is the proof                                                           |
| <code>new Promise((resolve, reject) =&gt; ...)</code>      | <code>Promise&lt;T&gt;</code>                          | the raw constructor                                                                |
| <code>Promise.resolve(v)</code>                            | <code>Promise&lt;T&gt;</code>                          | a settled ticket                                                                   |
| <code>Promise.reject(e)</code>                             | rejected <code>Promise</code>                          |                                                                                    |
| <code>Promise.all([...])</code>                            | <code>Promise&lt;T[]&gt;</code>                        | rejects on first failure                                                           |
| <code>Promise.allSettled([...])</code>                     | <code>Promise&lt;{status, value   reason}[]&gt;</code> | never rejects                                                                      |
| <code>Promise.race([...])</code>                           | <code>Promise&lt;T&gt;</code>                          | first to settle                                                                    |
| <code>Promise.any([...])</code>                            | <code>Promise&lt;T&gt;</code>                          | first to fulfil                                                                    |
| <code>p.then(f)</code>                                     | <code>Promise&lt;result of f&gt;</code>                | chaining always yields a Promise                                                   |
| <code>p.catch(f)</code>                                    | <code>Promise</code>                                   | same                                                                               |
| <code>p.finally(f)</code>                                  | <code>Promise</code>                                   | same                                                                               |
| <code>fetch(url, opts?)</code>                             | <code>Promise&lt;Response&gt;</code>                   | resolves on ANY HTTP status; rejects only on network failure                       |
| <code>res.json()</code>                                    | <code>Promise&lt;any&gt;</code>                        | the body, parsed                                                                   |
| <code>res.text()</code>                                    | <code>Promise&lt;string&gt;</code>                     |                                                                                    |
| <code>res.blob(), res.arrayBuffer(), res.formData()</code> | <code>Promise&lt;...&gt;</code>                        |                                                                                    |
| <code>import("./m")</code>                                 | <code>Promise&lt;Module&gt;</code>                     | dynamic import                                                                     |
| <code>crypto.subtle.*</code>                               | <code>Promise</code>                                   | Web Crypto                                                                         |
| <code>navigator.clipboard.readText() / writeText()</code>  | <code>Promise</code>                                   | browser                                                                            |
| <code>structuredClone</code>                               | a value, NOT a Promise                                 | listed to prevent the mistake                                                      |
| <code>JSON.parse, JSON.stringify</code>                    | a value, NOT a Promise                                 | synchronous; throw/return at once                                                  |
| <code>setTimeout(fn, ms)</code>                            | a timer id, NOT a Promise                              | becomes one only when wrapped: <code>new Promise(r =&gt; setTimeout(r, ms))</code> |

## Node built-ins

| Call                                                                               | Returns                           | Notes                                                                |
|------------------------------------------------------------------------------------|-----------------------------------|----------------------------------------------------------------------|
| <code>fs.promises.readFile, writeFile, readdir, stat, mkdir, unlink, access</code> | <code>Promise</code>              | <code>import { readFile } from "fs/promises"</code>                  |
| <code>fs.readFileSync</code> and every <code>*Sync</code>                          | a value                           | synchronous; blocks                                                  |
| <code>fs.readFile(path, cb)</code>                                                 | <code>undefined</code>            | callback style, not a Promise; wrap with <code>util.promisify</code> |
| <code>util.promisify(fn)</code>                                                    | a function that returns a Promise |                                                                      |
| <code>stream/promises pipeline, finished</code>                                    | <code>Promise</code>              |                                                                      |
| <code>timers/promises setTimeout(ms)</code>                                        | <code>Promise</code>              | the promisified sleep                                                |
| <code>child_process exec</code> via <code>promisify</code>                         | <code>Promise</code>              | plain <code>exec</code> is callback style                            |
| <code>dns.promises.*</code>                                                        | <code>Promise</code>              |                                                                      |
| <code>events.once(emitter, "name")</code>                                          | <code>Promise</code>              |                                                                      |
| <code>http.request</code>                                                          | a request object, NOT a Promise   | event-based                                                          |

## Database and HTTP clients (every one of these)

| Library                           | Returns a Promise from                                                                |
|-----------------------------------|---------------------------------------------------------------------------------------|
| pg                                | client.query(...), pool.query(...), pool.connect()                                    |
| mysql2/promise                    | conn.query(...), conn.execute(...)                                                    |
| prisma                            | prisma.user.findMany(), .findUnique(), .create(), .update(), .delete(), \$transaction |
| mongoose / mongodb                | Model.find(), .findOne(), .save(), collection.insertOne(), .updateOne()               |
| knex                              | every query builder chain once awaited; .then -able                                   |
| typeorm / sequelize / drizzle     | repository and model methods                                                          |
| ioredis / redis                   | client.get(), .set(), .del(), .incr(), .expire()                                      |
| axios                             | axios.get(), .post(), axios(config) → Promise<AxiosResponse>                          |
| node-fetch, undici, got, ky       | same shape as fetch                                                                   |
| @aws-sdk/* v3                     | client.send(command)                                                                  |
| firebase / firestore              | getDoc(), setDoc(), addDoc(), getDocs()                                               |
| stripe, twilio, sendgrid, any SDK | nearly every method                                                                   |

## Frameworks

| Context                                                    | Promise?                                                                                                                   |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Express app.get("/", async (req, res) => ...)              | the handler returns a Promise; Express 4 ignores it, so errors thrown inside are NOT caught without next(err) or a wrapper |
| Express res.json(), res.send(), res.status()               | NOT Promises; synchronous, chainable; and they do not stop the function                                                    |
| Next.js getServerSideProps, route handlers, server actions | async; return Promises                                                                                                     |
| Jest/Vitest it("...", async () => ...)                     | the test runner awaits it                                                                                                  |
| React useEffect(async () => ...)                           | × not allowed; useEffect expects a sync function or a cleanup; wrap an async call inside                                   |
| Event handlers onClick={async () => ...}                   | allowed; the Promise is ignored, so add .catch                                                                             |

## Your own code

| Shape                                             | Promise?                                                            |
|---------------------------------------------------|---------------------------------------------------------------------|
| async function f() {}                             | yes                                                                 |
| const f = async () => {}                          | yes                                                                 |
| class A { async m() {} }                          | yes                                                                 |
| function f() { return fetch(...) }                | yes, pass-through                                                   |
| function f() { return someAsyncFn() }             | yes, pass-through                                                   |
| function f() { someAsyncFn() } with no return     | NO: returns undefined; the Promise is dropped (fire-and-forget bug) |
| function f() { return [1, 2].map(x => x) }        | no                                                                  |
| function f(): Promise<void> { ... } without async | must return a Promise in the body or it will not compile            |

## Things that look async and are not

| Looks like                  | Actually                                                                           |
|-----------------------------|------------------------------------------------------------------------------------|
| JSON.parse(s)               | synchronous; throws immediately on bad input                                       |
| arr.map(async x => ...)     | map itself is synchronous and returns Promise[]; the Promises are inside the array |
| arr.forEach(async x => ...) | forEach is synchronous and returns undefined; the Promises are thrown away         |
| setTimeout(fn, 0)           | schedules; returns an id; not awaitable                                            |
| localStorage.getItem        | synchronous                                                                        |
| fs.readFileSync             | synchronous                                                                        |
| res.json(data) in Express   | synchronous send; nothing to await                                                 |
| console.log                 | synchronous                                                                        |

| Looks like                       | Actually                                             |
|----------------------------------|------------------------------------------------------|
| <code>Date.now()</code>          | synchronous                                          |
| <code>crypto.randomUUID()</code> | synchronous ( <code>crypto.subtle.*</code> is async) |

## Decision procedure for any call `g(...)`

1. Is `g` declared `async`? → Promise. Stop.
2. Does its declaration say `: Promise<...>?` → Promise. Stop.
3. Is `g` on the Promise-makers list above (`fetch`, `db.query`, `.then`, `new Promise`, `fs/promises`, any client SDK)? → Promise. Stop.
4. Open `g`'s body. Find every `return`. Is any returned expression a call that passes steps 1–3? → Promise. Stop.
5. Is `g` awaited or `.then`'d anywhere else in the file? → the author thinks it is a Promise; treat it as one and verify with 1–4.
6. None of the above → a plain value.

Then, for every call site of `g`: if it is a Promise, is it `await`ed, `returned`, `.then`'d, or inside `Promise.all`? If not, that is the finding.

## What the type says once you know

| Written                                         | What you hold after <code>await</code>                                                      |
|-------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>Promise&lt;number&gt;</code>              | a number                                                                                    |
| <code>Promise&lt;User&gt;</code>                | a User object                                                                               |
| <code>Promise&lt;User[]&gt;</code>              | an array of Users                                                                           |
| <code>Promise&lt;User   null&gt;</code>         | a User or null; check before <code>.name</code>                                             |
| <code>Promise&lt;void&gt;</code>                | nothing useful; the point was the side effect                                               |
| <code>Promise&lt;Response&gt;</code>            | a fetch Response; still need <code>res.ok</code> and <code>await res.json()</code>          |
| <code>Promise&lt;any&gt;</code>                 | anything; checking is off (what <code>res.json()</code> gives you)                          |
| <code>Promise&lt;unknown&gt;</code>             | anything; must narrow before use                                                            |
| <code>Promise&lt;User&gt;[]</code>              | not awaitable directly; <code>await Promise.all(...)</code> first, then <code>User[]</code> |
| <code>Promise&lt;Promise&lt;User&gt;&gt;</code> | does not exist; Promises flatten; it is just <code>Promise&lt;User&gt;</code>               |